

TWiN Term Project Report

Project Title: Using TelecomRAG for Analysis for 5G MAC Logs

Team Members (Names & Roll Numbers):

Dhanraj Shrivastav (CS24MTECH11004)

Abstract

A brief summary of problem, approach, results, and contributions (150–250 words).

Evaluation Criteria: Clarity, completeness, and ability to summarize contribution succinctly.

Problem summary: LLMs are tools to perform analysis of 5G logs, including root cause identification. However, they suffer from prompt dependency. Generic prompts from novices yield poor diagnostics, whereas expert targeted prompts produce precise causal chains (see *LLM-5GMAC*, <https://dl.acm.org/doi/pdf/10.1145/3737900.3770164>). To bridge this skill gap, we explore Retrieval-Augmented Generation (RAG) alongside classical NLP and sentence embedding-based key event extraction.

Approach: We evaluated 13 distinct pipeline configurations for two log files. We compared two broad strategies:

1. **RAG:** Using 3GPP TS 38.321 specifications and OAI MAC documentation in the context window.
2. **Key event extraction:** Using statistics/classical NLP and embedding approach to extract key events from the logs, which are fed to the LLM to guide the RCA.

Results: Our key findings are:

- **RAG is ineffective for deep RCA:** Generic prompts fail to reveal causal chains even with RAG. Furthermore, RAG adds negligible value to prompts that already contain extracted events.
- **LLMs struggle with raw logs:** General-purpose LLMs fail at zero-shot log translation and extraction, frequently generating inaccuracies.
- **NLP and embeddings beat LLMs for extraction:** Classical NLP and sentence embedding methods accurately isolate key events from log noise.
- **Targeted prompting wins:** Prompts enriched with extracted key events vastly outperform RAG-based approaches.

Contributions: We propose using extracted key events to eliminate prompt dependency in 5G log analysis. By proving the inefficacy of RAG for RCA, we conclude future efforts should focus on building specialized LLM agents dedicated to precise key event extraction.

Introduction / Project Recap

Problem statement, motivation, objectives, and project type.

Evaluation Criteria: Clarity of problem, relevance, and well-defined objectives.

Problem statement: Explore the use of RAG based on relevant documents in eliminating prompt dependency in 5G MAC log analysis.

Motivation: To close the skill gap between novice and experienced operators in performing root cause analysis from MAC logs.

Objectives:

- Mitigate prompt dependency from 5G MAC log analysis.
- Explore the use of RAG in 5G MAC log analysis. Assess the efficacy of RAG in accomplishing the above objective
- Explore alternative data-driven approaches to mitigate prompt dependency and close the operator skill gap.
- Compare and contrast the different approaches and assess the efficacy of each one.

Project Type: Comparative Study

Related Work and Positioning

Discuss 2–4 related works and highlight your contribution/gap.

Evaluation Criteria: Depth of understanding, correctness of comparison, and clarity of positioning.

Related Works:

1. *LLM-5GMAC: Performance Optimization in O-RAN Split 7.2 Using LLM-Based MAC-Layer Log Analysis* (<https://dl.acm.org/doi/10.1145/3737900.3770164>)

This work highlights the prompt dependency problem. Novice operators use generic prompts simply asking the LLM to analyze the given logs resulting in poor diagnostic quality of the response generated. By spotting and understanding key anomalous events in the logs (invalid CQI error, invalid TA offset, excessive HARQ NACKs, etc.), one can create a targeted

prompt which enables the LLM to generate a precise, multi-layer causal chain. However, identification of such events requires operator expertise. The work highlights the need to close this skill gap.

2. TelecomRAG: Taming Telecom Standards with Retrieval Augmented Generation and LLMs (<https://dl.acm.org/doi/10.1145/3711992.3711996>)

This paper builds a question-answering system for 5G related user queries using retrieval augmented generation (RAG) over 3GPP specifications. The system gave better answer and more detailed answers to user queries, citing relevant spec documents than LLMs like ChatGPT-4, Gemini Ultra and TelecomGPT. This paper suggests using RAG to enrich a prompt with additional context that may help in root cause analysis.

My contribution:

13-mode pipeline: Built a modular pipeline to compare the impact of RAG, zero-shot event extraction, NLP-based key event extraction with hardcoded keywords and chi-squared learned keywords, and sentence embedding based key event extraction.

Critique of RAG for RCA: Provided empirical evidence that standard RAG techniques fail to improve causal reasoning in log analysis.

Superiority of Classical NLP: Demonstrated that data-driven techniques (Chi-square keyword extraction, sentence embedding and heuristic parsing) are better than LLMs at extracting key anomalous events.

Prompt Dependency Mitigation: Demonstrated that enriching prompts with key events rather than vector databases produces more accurate and detailed root cause analysis.

System Model / Threat Model

Describe system components, assumptions, constraints, and threat model.

Evaluation Criteria: Completeness, correctness, and realism of assumptions.

System components:

1. Log generation source: OAI setup using USRP B210s, ns-3 harq exhaustion scenario; ns-3 logging header that implements logging callbacks to 5G module trace sources.

2. Event extraction module: Drain3 is used to parse logs to convert them into log templates. Then key / anomalous event extraction is done using the following different methods:

a. Scoring against a hardcoded set of anomalous keywords.

b. Scoring against a set of anomalous keywords learned from log files (data-driven).

This also involves determining labels for each log template (NORMAL/ANOMALOUS).

Used LLM with few-shot prompting here. Uses Chi-square scoring to identify keywords that skew towards the ANOMALOUS class.

c. Passing raw logs to embedding model to obtain sentence embeddings and identifying small clusters in embedding space.

3. **Knowledge base:** 3GPP TS38.321 (MAC protocol specification), mac-usage.md from OAI.

4. **Retrieval engine:** RAG implementations that use two different kinds of queries:

a. Concatenate the extracted key anomalous events and use them for retrieval.

b. Use LLM to convert raw log to a short natural language description of key chain of events in the log and use that as retrieval query.

5. **Large Language Model:** One local LLM needed for log analysis. Used gemma-4 for this. Another needed for classification for scoring and for converting raw logs to natural language query.

Assumptions:

1. The logs are such that they actually do contain identifiable key events. Subtle metric changes will be difficult to identify and consider as key events.
2. For the sentence embedding approach, assumption is that the key events are rarer than normal events and tend to form smaller clusters.
3. The few-shot classification of anomalous log templates by the LLM is accurate. The significant anomalous keyword learning is dependent on this.
4. Zero-shot conversion of raw logs to natural language is accurate (this assumption breaks in practice, motivating use of data-driven NLP approaches like the significant words approach).

Constraints:

1. **Context Window Limitation and Rate Limits:** Logs had to be trimmed before being sent to Groq due to tokens per minute limitation of the free tier.
2. **Inference latency:** Locally running models had slower inference speeds.

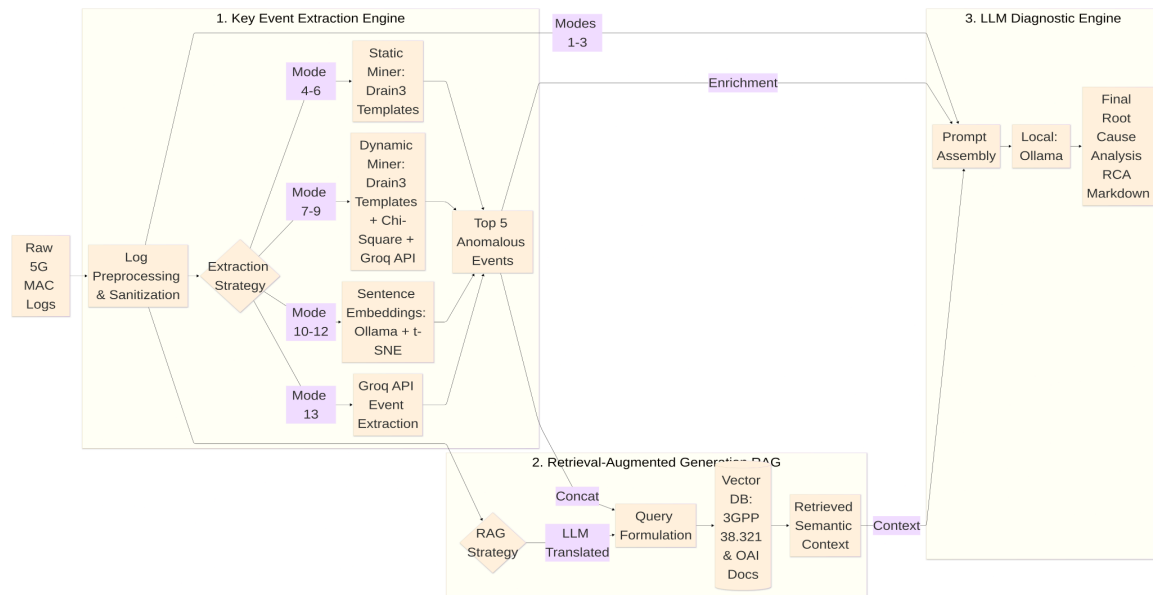
Threat Model: (Not Applicable here)

System Architecture and Methodology

Include architecture diagram, modules, and data/control flow.

Evaluation Criteria: Clarity of design, logical flow, and originality of contribution.

Architecture Diagram/System Flow Graph:



Modules:

The architecture has the following modules:

Log Preprocessing Module: Raw logs generated by the OpenAirInterface (OAI) MAC layer contain noise, ANSI escape codes, and highly variable runtime values. This module uses regex to sanitize the data and mask volatile parameters (e.g., standardizing CQI 0 to <CQI_INVALID>). This transforms the unstructured text into highly structural data, preparing it for both mathematical analysis and LLM ingestion.

Key Event Extraction Engine (Core Contribution): This engine acts as the primary filter to identify the top 5 most critical anomalous events from the log noise. To comprehensively benchmark extraction efficacies, it features four interchangeable extraction mechanisms:

Static Miner: Utilizes the Drain3 parsing tree to extract log templates and scores them based on heuristic rarity and hardcoded vocabulary.

Dynamic Miner: Employs Chi-Square statistical testing to organically learn a vocabulary of significant terms from the logs, using this localized vocabulary to dynamically score the log templates obtained from Drain3.

Sentence Embedding & Clustering: Converts log lines into high-dimensional vectors (via qwen3-embedding:4b) and applies Agglomerative Clustering and takes events corresponding to small clusters.

Groq API Event Extraction (mode13_pipeline.py): A zero-shot approach. It directly prompts the Groq API to read the sanitized logs and extract the key events.

Retrieval-Augmented Generation (RAG) Module: A LangChain-powered vector retrieval system designed to pull standard-compliant diagnostic context. Depending on the pipeline configuration, it builds queries either by concatenating the extracted events or via an LLM-powered semantic translation of the raw logs. It then queries an embedded knowledge base consisting of 3GPP TS 38.321 specifications and OAI usage documents to retrieve the top-K most relevant semantic chunks.

LLM Diagnostic Engine: The final synthesis and reasoning layer. It compiles the sanitized logs, the top 5 anomalous events (extracted from Module 2), and the retrieved RAG documentation (from Module 3) into a strictly formatted prompt. A reasoning LLM (e.g., local Ollama models or the Groq API) processes this explicitly enriched prompt to logically trace the causal chain of the network failure and output a formatted Root Cause Analysis (RCA) Markdown artifact.

Modes1-3: Baseline LLM only gets RAG context using two possible RAG methods: concat and LLM translated. No Enrichment here.

Core Algorithm / Design Logic

Provide pseudo-code/flowchart and explanation of logic.

Evaluation Criteria: Technical depth, correctness, and clarity of explanation.

Pseudo-code for overall pipeline:

```
// 1. Preprocessing
FUNCTION Preprocess(raw_logs):
    clean_logs = Remove_ANSI_Codes(raw_logs)
    clean_logs = Regex_Mask_Variables(clean_logs) // e.g., Replace specific CQI with <CQI_VAL>
    RETURN clean_logs

// 2. Extraction Strategies (Returns Top 5 Events)
FUNCTION Extract_Static(clean_logs):
    parsed_tree = Drain3_Parse(clean_logs)
    scored_events = Score_By_Heuristics(parsed_tree)
    RETURN Top_N(scored_events, N=5)

FUNCTION Extract_Dynamic(clean_logs):
    learned_vocab = Chi_Square_Test(clean_logs, normal_corpus)
    scored_events = Score_By_Vocab(clean_logs, learned_vocab)
    RETURN Top_N(scored_events, N=5)

FUNCTION Extract_Embeddings(clean_logs):
    vectors = Qwen3_Embed(clean_logs)
    clusters = Agglomerative_Clustering(vectors)
    outliers = Isolate_Sparse_Clusters(clusters)
    RETURN Top_N(outliers, N=5)
```

```

FUNCTION Extract_Groq(clean_logs):
    prompt = "Extract top 5 anomalies from these logs:" + clean_logs
    RETURN Groq_API_Call(prompt)

// 3. RAG Retrieval Strategies
FUNCTION RAG_Concat(extracted_events, vector_db):
    query = Join_Strings(extracted_events)
    RETURN vector_db.Search(query, top_k=3)

FUNCTION RAG_Translated(clean_logs, vector_db):
    translation_prompt = "Translate these MAC logs into a plain English search query."
    semantic_query = Groq_API_Call(translation_prompt + clean_logs)
    RETURN vector_db.Search(semantic_query, top_k=3)

// 4. LLM Reasoning Engine
FUNCTION LLM_Analyze(clean_logs, extracted_events, rag_context, prompt_type):
    prompt = Build_Template(prompt_type)
    prompt.add(clean_logs)
    IF extracted_events IS NOT NULL: prompt.add("Focus on these events:", extracted_events)
    IF rag_context IS NOT NULL: prompt.add("Reference this 3GPP context:", rag_context)

    rca_report = Reasoning_LLM_Call(prompt)
    RETURN rca_report

```

Pseudo-code for Static Mining of Key Events:

```

// 1. Static Miner: Heuristic Scoring Routine
FUNCTION Score_By_Heuristics(template_freq, ordered_templates, log_lines):
    total_lines = Length(log_lines)
    first_occurrence = EMPTY_MAP

    // Step 1: Temporal Mapping
    // Identify the first line index where each template appears.
    // Logic: Errors appearing earlier in the sequence are more likely to be the root cause.
    FOR index, template IN ordered_templates:
        IF template NOT IN first_occurrence:
            first_occurrence[template] = index

    // Step 2: Vocabulary Initialization
    // Define and stem hardcoded telecom/network error keywords.
    significant_terms = ["fail", "invalid", "loss", "drop", "timeout", "exceed", "mismatch", "unknown", "abort", "reject", "release", "disconnect", "retx",
        "nack", "corrupt", "failure"]
    significant_stems = Stem_Words(significant_terms)

    scores = EMPTY_MAP

    // Step 3: Composite Scoring Matrix
    FOR template, count IN template_freq:

        // A. Rarity Weighting (Range: ~0.0 to 1.0)
        // Highly frequent logs (noise) approach 0; rare anomalies approach 1.
        rarity_score = 1 - (count / total_lines)

        // B. Semantic Significance Weighting
        // Tokenize the template, stem the words, and count matches against the hardcoded error dictionary.
        template_stems = Stem_Words(Tokenize(template))
        significance_score = Count_Overlaps(template_stems, significant_stems)

        // C. Temporal Weighting (Range: 0.0 to 1.0)
        // Templates appearing at the top of the log approach 1; end of the log approach 0.
        position = first_occurrence[template]
        temporal_score = 1 - (position / total_lines)

        // D. Final Formula Calculation
        // Adds 1 to semantic and temporal scores to prevent zero-multiplication nullification.
        scores[template] = rarity_score * (significance_score + 1) * (1 + temporal_score)

```

```
// Step 4: Output
RETURN Sort_Descending(scores)
```

Pseudo-code for Dynamic Mining of Key Events:

```
// 1. Dynamic Miner: Chi-Square Vocabulary Learning
FUNCTION Learn_Significant_Vocab(classification_map):
    // Extract log templates and map their string labels to binary values
    templates = Extract_Keys(classification_map)
    labels = Convert_To_Binary(classification_map) // 1 if ANOMALOUS, 0 if NORMAL

    // Fallback: If the LLM classified everything the same, statistical variance is impossible
    IF All_Labels_Identical(labels):
        RETURN EMPTY_SET

    // Step 1: Vectorization
    // Convert templates into a bag-of-words feature matrix, ignoring stop words
    feature_matrix, vocabulary = Vectorize_Text(templates)

    // Step 2: Statistical Calculation
    // Calculate the Chi-Square correlation between each word and the ANOMALOUS label
    chi2_scores = Calculate_Chi_Square(feature_matrix, labels)

    // Calculate how often each word appears in each respective class
    anomalous_counts = Sum_Frequencies_If(feature_matrix, labels == 1)
    normal_counts = Sum_Frequencies_If(feature_matrix, labels == 0)

    // Step 3: Dynamic Filtering
    dynamic_vocab = EMPTY_LIST

    FOR index, term IN vocabulary:
        score = chi2_scores[index]

        // Condition A: Must pass the baseline statistical significance threshold (> 0.5)
        // Condition B: Must organically appear MORE in anomalous logs than in normal logs
        IF score > 0.5 AND anomalous_counts[index] > normal_counts[index]:
            dynamic_vocab.Append(term)

    RETURN dynamic_vocab

// 2. Dynamic Miner: Template Scoring Routine
FUNCTION Score_By_Dynamic_Vocab(template_freq, ordered_templates, log_lines, learned_vocab, classification_map):
    total_lines = Length(log_lines)
    first_occurrence = EMPTY_MAP

    // Step 1: Temporal Mapping
    // Identify the first occurrence index (Note: Calculated here, but excluded from final weight)
    FOR index, template IN ordered_templates:
        IF template NOT IN first_occurrence:
            first_occurrence[template] = index

    scores = EMPTY_MAP

    // Step 2: Composite Scoring Matrix
    FOR template, count IN template_freq:
        // A. LLM Classification Weighting
        // Heavily bias the score based on the initial LLM labeling
        label = classification_map[template]
        IF label == "ANOMALOUS":
            label_weight = 10.0
        ELSE:
            label_weight = 0.01

        // B. Rarity Weighting (Range: ~0.0 to 1.0)
        rarity_score = 1 - (count / total_lines)
```



```
// C. Semantic Significance Weighting
// Count overlaps between the template words and the dynamically learned Chi-Square vocabulary
template_words = Tokenize_And_Lowercase(template)
significance_score = Count_Overlaps(template_words, learned_vocab)

// D. Final Formula Calculation
// Calculation applies rarity, dynamic vocabulary overlap, and heavy LLM label weighting
scores[template] = rarity_score * (significance_score + 1) * label_weight

// Step 3: Output
RETURN Sort_Descending(scores)
```

Different Modes of Operation:

By passing different parameters into the core functions, the orchestrator script dynamically executes the 13 modes.

Baseline Modes (No Event Extraction provided to the LLM)

- **Mode 1 (Pure Baseline):** Analyze(logs, null, null, "Baseline")
- **Mode 2 (Baseline + Concat RAG):** *Note: Uses Static Miner strictly behind the scenes to build the RAG query, but does NOT feed the extracted events to the LLM's prompt.* hidden_events = Extract_Static(logs) ctx = RAG_Concat(hidden_events) Analyze(logs, null, ctx, "Baseline")
- **Mode 3 (Baseline + LLM Translated RAG):** ctx = RAG_Translated(logs) Analyze(logs, null, ctx, "Baseline")

Static Miner Modes (Drain3 Heuristics)

- **Mode 4 (Static + No RAG):** events = Extract_Static(logs); Analyze(logs, events, null, "RCA")
- **Mode 5 (Static + Concat RAG):** events = Extract_Static(logs); ctx = RAG_Concat(events); Analyze(logs, events, ctx, "RCA")
- **Mode 6 (Static + LLM Translated RAG):** events = Extract_Static(logs); ctx = RAG_Translated(logs); Analyze(logs, events, ctx, "RCA")

Dynamic Miner Modes (Chi-Square Statistics)

- **Mode 7 (Dynamic + No RAG):** events = Extract_Dynamic(logs); Analyze(logs, events, null, "RCA")
- **Mode 8 (Dynamic + Concat RAG):** events = Extract_Dynamic(logs); ctx = RAG_Concat(events); Analyze(logs, events, ctx, "RCA")
- **Mode 9 (Dynamic + LLM Translated RAG):** events = Extract_Dynamic(logs); ctx = RAG_Translated(logs); Analyze(logs, events, ctx, "RCA")

Embedding & Clustering Modes

- **Mode 10 (Embeddings + No RAG):** events = Extract_Embeddings(logs); Analyze(logs, events, null, "RCA")
- **Mode 11 (Embeddings + Concat RAG):** events = Extract_Embeddings(logs); ctx = RAG_Concat(events); Analyze(logs, events, ctx, "RCA")
- **Mode 12 (Embeddings + LLM Translated RAG):** events = Extract_Embeddings(logs); ctx = RAG_Translated(logs); Analyze(logs, events, ctx, "RCA")

Native LLM Extraction Mode

- **Mode 13 (Groq Extraction + No RAG):** events = Extract_Groq(logs); Analyze(logs, events, null, "RCA")

Explanation of Logic:

- 1. Drain3 Log Parsing (Static Miner):** Logs are highly repetitive. Drain3 parses raw log lines into templates (e.g., replacing metrics by <*> in the log lines).

There is a hardcoded set of significant words = ["fail", "invalid", "loss", "drop", "timeout", "exceed", "mismatch", "unknown", "abort", "reject", "release", "disconnect", "retx", "nack", "corrupt", "failure"]

significance_score = The number of significant words in a template.

rarity_score = 1 - (# of times log lines of this template) / (total number of log lines)

[weights rarer templates higher]

temporal_score = 1 - (first position in which this template appears) / (total number of log lines)

[weights root causes higher as they tend to appear earlier]

$$final_score\ for\ template = rarity_score * (1 + significance_score) * (1 + temporal_score)$$

The top 5 templates according to this score are the key anomalous events.

2. **Chi-Square Testing (Dynamic Miner):** Instead of hardcoding the significant word list. This algorithm simply learns this list from the corpus. This allows the system to generalize to new unseen anomalies.

First, an LLM (llama-3.3-40b-versatile) is used to classify Drain3 templates into NORMAL or ANOMALOUS using few-shot prompting.

This gives us two classes of templates. Extract all words from all the templates. Now, a chi-squared test is applied to find the words that skew towards the ANOMALOUS class. This list of words is the learned significant word list.

For example, for invalid TA offset related causal chain, the learned significant words are:
*[timing,ignore,offset,ulsch,reported,advance,unexpected,detected,proc,pid,header,invalid,warni
ng,unknown,dtx,infeasible,harq,residual,length]*

For harq exhaustion because of weak link the learned list is:
[sched,stall,mcs,feedback,ri,nack,b,cqi,nr,scheduling,tbs,wb,harq,exhausted]

The rest of the scoring is done as in static mining as:

$rarity_score * (1 + significance_score) * label_weight$

$label_weight = 10$ if $label=ANOMALOUS$, 0.01 if $label=NORMAL$ (discards normal templates)

3. **Agglomerative Clustering (Embeddings):** Sentences are converted into dense vector representations. Then agglomerative clustering with $k=15$ clusters is done. The five smallest clusters and the majority log lines in them are taken as the key events.
4. **LLM as event extractor:** The cleaned up logs are given to a large LLM *llama-3.3-70b-versatile*. It is prompted to extract 5 key events from these logs.

Implementation Details

Hardware, software tools, implementation details, challenges.

Evaluation Criteria: Depth of implementation, clarity on what was built vs reused, and problem-solving.

Hardware: USRP B210s to collect du_mac_logs_3.txt, NVIDIA A6000 GPU for local LLM inference.

Software Tools: Ollama, LangChain, ChromaDB, Drain3, Scikit-learn, Plotly, Groq API, Docling

Implementation Details:

- Reused components:

- The base LLMs (via Groq and Ollama) and their underlying weights.
- Drain3 parsing algorithm.
- Standard Agglomerative Clustering and Chi-Square mathematical functions provided by `scikit-learn`.
- LangChain's baseline document loaders and vector retrieval functions.
- Built components:
 - __Script *full_pipeline.py* that runs modes 1-12 and outputs all results cleanly in results directory.
 - __Script *mode13_pipeline.py* that does the same for mode 13.
 - __BM25+Vector Search Hybrid RAG in the source file *retrieval_pipeline.py*
 - __PDF to Markdown conversion using *docling* preserving table structure *convert_to_markdown_docling.py*.
 - __Data ingestion pipeline using ChromaDB and *qwen3-embedding:4b*
 - __Sentence clustering using sklearn *AgglomerativeClustering* and *qwen3-embedding:4b* sentence embeddings
- Other implementation details:
 - Caching of all results produced using Groq API. To ensure lower API credit consumption. This includes template classification results, natural language query translations.
 - Saving of retrieved chunks for each RAG mode.
 - Config files used to set parameters instead of hardcoded paths.

Experimental Setup

Scenario design, parameters, metrics, baselines, reproducibility.

Evaluation Criteria: Rigor of experimental design and reproducibility.

Scenario design:

- Disconnect one USRP DU and USRP UE by moving them apart (du_mac_logs_3.txt)
- ns-3 scenario: (harq_exhaustion_logs_new.txt)

Models a stressed 5G NR downlink scenario in which a single UE communicates with a gNB over a 3GPP UMa channel at 3.5 GHz. A continuous UDP stream is sent from a remote host to the UE, placing sustained load on a deliberately degraded radio link. As path loss and noise drive repeated transmission failures, the gNB MAC exhausts its four-round HARQ retransmission limit, causing the scheduler to stall and throughput to collapse. The simulation traces this cascading failure across layers — from HARQ exhaustion at the MAC, through potential RRC context release — replicating the behavior of a cell-edge UE experiencing progressive link degradation in a 5G NR network.

Parameters:

General Pipeline Parameters

- GROQ_LLM_MODEL=llama3.3-70b-versatile
- GROQ_LLM_TEMPERATURE=0.0
- LOCAL_LLM_MODEL=gemma4:latest
- LOCAL_LLM_TEMPERATURE=0.0

Retrieval-Augmented Generation (RAG)

- RETRIEVER_TYPE=EnsembleRetriever
- VECTOR_WEIGHT=0.7
- BM25_WEIGHT=0.3
- COMPONENT_RETRIEVAL_K=5
- FINAL_CONTEXT_CAP=5
- VECTOR_SIMILARITY_METRIC=cosine
- SCORE_THRESHOLD=0.3

Static Miner (Drain3)

- DRAIN_SIMILARITY_THRESHOLD=0.4

Dynamic Miner (Chi-Square & LLM)

- NGRAM_RANGE=(1, 1)
- TOKEN_PATTERN=\b[a-z]+\b
- CHI_SQUARE_THRESHOLD=0.5
- ANOMALOUS_MULTIPLIER=10.0

- NORMAL_MULTIPLIER=0.01

Sentence Embeddings & Clustering

- EMBEDDING_MODEL=qwen3-embedding:4b
- CLUSTERING_ALGORITHM=AgglomerativeClustering
- CLUSTERING_LINKAGE=ward
- NUM_CLUSTERS=15
- TSNE_COMPONENTS=2
- TSNE_RANDOM_STATE=42
- TSNE_PERPLEXITY=30

Results and Analysis

Plots/tables, comparisons, explanation of results.

Evaluation Criteria: Quality of analysis, insightfulness, and correctness of interpretation.

The following are the observations made:

1. Generic prompt does not reveal causal chain, even with RAG (both forms concat+LLM translation)

Evidence to support: None of the general prompt outputs (modes 1,2,3 with concat rag, llm_translation rag) gave any meaningful causal chain analysis.

Kindly see files 'results/du_mac_logs_3/mode1/mode1_analysis_wo_rag.md',
'results/du_mac_logs_3/mode2/mode2_analysis_with_rag_concat.md',
'results/du_mac_logs_3/mode3/mode3_analysis_with_rag_llm.md'

Also see the corresponding files in 'results/harq_exhaustion_logs_new_trimmed/' as well.

Below is a screenshot (second half cut off due to size, please see the markdown in the repo for full output) of one of the files (with rag_llm analysis of du_mac_logs_3.txt):

```

This analysis focuses on the provided MAC layer logs, which contain detailed metrics regarding signal strength, data throughput, error rates, and synchronization status.

## 📶 Wireless Link Quality Analysis

Overall, the logs indicate a generally stable and high-throughput connection for the UE (RNTI: ff04), but they also reveal several recurring synchronization and resource allocation warnings that suggest potential underlying physical layer or configuration issues.

## 🟢 Strengths (Good Performance Indicators)

1. High Throughput: The data transfer rates are excellent.
   * MAC Throughput: The 'MAC_TX' and 'RX' bytes consistently increase over time (e.g., from 73894 to 73866533 TX, and 66724 to 1870958 RX).
   * LCID 4 Throughput: The primary data channel (LCID 4) shows massive, sustained data transfer (e.g., 1,093,596 bytes at Slot 768.0, 1,537,230 bytes at Slot 384.0).
2. Low Error Rates (BER): For the majority of the session, the Block Error Rate (BER) is extremely low, often reaching $0.000005 or $0.000015. This indicates that the physical layer is successfully decoding most transmitted data.
3. Stable RSRP: The Received Signal Strength Indicator (RSRP) remains strong and stable, fluctuating between -84 dB and -95 dB, which is typical for a good cellular connection.
4. Successful RA Procedure: The Initial Random Access (RA) procedure for 'ff04' was successful, confirming initial synchronization.

## 🟡 Weaknesses and Areas of Concern (Potential Issues)

1. Timing Advance (TA) Warnings (Critical):
   * The most frequent and concerning warning is: 'Invalid timing advance offset for RNTI ff04'.
   * Impact: Timing Advance (TA) is crucial for synchronizing the UE's uplink transmission timing with the gNB. Repeated "Invalid timing advance offset" warnings suggest that the UE is either calculating its timing incorrectly or the gNB is receiving timing information that is inconsistent or invalid. This can lead to intermittent uplink transmission failures or reduced capacity.
2. BLER Spikes (Moderate):
   * While most BER values are near zero, there are noticeable spikes, particularly in the later slots (e.g., $0.127075, $0.137425, $0.165865).
   * Impact: These spikes indicate periods where the link quality degrades, leading to a higher percentage of corrupted data blocks.
3. Resource Utilization Imbalance (Minor):
   * The logs show that 'LCID 2' consistently reports 'Tx 0 RX 0 bytes'.
   * Impact: If LCID 2 is intended for a specific service or control channel, the fact that it is never used suggests either a configuration mismatch or that the service associated with this logical channel is inactive.
4. RA Procedure Failures (Transient):
   * The logs show multiple instances of the UE failing to schedule PUSCH (e.g., 'UE ff04: Detected UL Failure on PUSCH after 10 PUSCH DTX, stopping scheduling').
   * Impact: This indicates the UE is repeatedly failing to maintain uplink scheduling, which is a symptom of potential timing, power, or resource allocation issues.

```

Explanation: Simply retrieving more information about 5G MAC specs does not help the LLM reason deeply about root causes.

Note: The LLM above is gemma4 for local inference on the logs.

2. LLM translated retrieval query built directly from logs is either incorrect or nonsensical.

Evidence to support:

shared_translated_query.txt for du_mac_logs_3.txt is:

‘invalid timing advance offset and ul failure on pusch after dtx’

This is incorrect. The correct order of events is: ‘invalid timing advance offset causes ul failure on pusch and dtx’

shared_translation_query.txt for harq_exhaustion_logs_new_trimmed.txt is:

‘harq nack scheduling stall mcs invalid cqi feedback sinr dl ctrl dl data’

which is not a meaningful English phrase.

Explanation: The LLM cannot accurately determine the root cause event unless a more targeted prompt is given.

3. LLM extracted top key events are often inaccurate/unhelpful:

Evidence to support:

In *results/harq_exhaustion_logs_new_trimmed/mode13/mode13_key_events_wo_rag.txt*, we have:

1. [MAC][0.404500] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 0
2. [MAC][0.405500] DL HARQ | RNTI 1 | PID 15 | NACK | Retx 1

3. [MAC][0.407000] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 1
4. [MAC][0.410500] DL HARQ | RNTI 1 | PID 15 | NACK | Retx 3
5. [MAC][0.404500] DL HARQ | RNTI 1 | PID 12 | NACK | Retx 0 *** HARQ EXHAUSTED → scheduling stall

With mode7 (Dynamic miner + No RAG), more relevant events show up:

In *results/harq_exhaustion_logs_new_trimmed/mode7/mode7_key_events_wo_rag.txt*, we have:

1. <*> NR HARQ NACK <*> for RNTI=1 <*> *** HARQ EXHAUSTED → scheduling stall
2. [MAC <*> DL SCHED | Frame <*> Slot <*> | RNTI 1 | MCS <MCS_INVALID> | TBS 136 B | HARQ <*>
3. [PHY <*> CQI FEEDBACK | RNTI 1 | WB-CQI <CQI_INVALID> | MCS <MCS_INVALID> | RI 1
4. <*> NR HARQ NACK <*> for RNTI=1 <*>
5. [MAC <*> DL HARQ | RNTI 1 | PID <*> | <*> | Retx <*>

Explanation: These events are rare in the trimmed log files. The LLM probably does not focus on these fewer but regularly occurring events line HARQ NACKs. Also, it is not working with log templates like the dynamic miner approach which makes it hard to key events and instead focus on metric values.

Perhaps passing the templates to the LLM might work but that will destroy the temporal nature of the logs making it difficult for the LLM for answer based on a reasonable causal chain.

Also explicit significance scores from learned vocabulary boosts anomalies up in Mode 7.

4. A targeted prompt built using extracted key events performs much better than RAG in doing RCA.

Evidence to support:

All modes without RAG but with key event extraction and targeted prompt build a causal chain where general prompts fail to even with RAG.

Let us compare gemma:4 outputs of Mode 7 (Dynamic Miner + no RAG) with Mode 2 (Baseline + Concat RAG) on *du_mac_logs_3.txt*:

Mode 2: Screenshot cut off because of length of response (check the full file in *results/du_mac_logs_3/mode2/*)

```
1 This analysis focuses on the provided MAC layer logs, interpreting the key performance indicators (KPIs) and error messages against the provided 3GPP context documentation.
2
3 ## 📶 Wireless Link Quality Analysis
4
5 Overall, the logs demonstrate a **highly stable and high-throughput connection** for an extended period, indicating that the basic radio link establishment and data transfer mechanisms are functioning correctly. However, the latter half of the logs show a
6 **clear degradation in link quality and increased error rates**, suggesting potential environmental or configuration issues that need addressing.
7
8 ### 🟢 Strengths (what is working well)
9
10 1. **Successful Connection and Mobility**: The UE successfully completes the Random Access (RA) procedure and maintains connectivity across many slots (from Slot 384.0 up to Slot 896.0 in the final segment).
11 2. **High Throughput**: The MAC layer statistics show consistently high data transfer rates (e.g., TX reaching 18-11 Mbps in the later slots, and RX consistently above 1.2 MB/s). This indicates the physical layer and MAC scheduling are effectively
12 utilizing the allocated resources.
13 3. **Stable Signal Strength (RSRP)**: The average RSRP remains strong, generally fluctuating between -85 dBm and -95 dBm. While -95 dBm is approaching the limit, the consistency suggests the UE is maintaining a stable path loss profile.
14 4. **Low Initial Error Rates**: For the first several hundred slots, the BLER (Block Error Rate) is extremely low (e.g., 0.00080 to 0.00081), indicating excellent initial link quality.
15
16 ### 🟡 Weaknesses and Areas of Concern (Degradation and Errors)
17
18 The primary issues are concentrated in the latter half of the log file, suggesting the link quality is deteriorating over time or under sustained load.
19
20 #### 🔍 Increasing Error Rates (BLER and DTX)
21 1. **BLER Trend**: The BLER (Block Error Rate) shows a clear upward trend. It starts near zero (e.g., 0.00080) and gradually increases, reaching values like **0.16580** (Slot 384.0, final segment) and **0.13742** (Slot 384.0, final segment).
22    * **Interpretation**: A BLER of 0.1 to 0.2 is significantly higher than the ideal target (typically < 0.1, or even < 0.05 for high-reliability services). This indicates that a substantial percentage of transmitted data blocks are corrupted or lost,
23    leading to retransmissions and reduced effective throughput.
24 2. **Missed Errors / Missed Errors**: The number of errors in both DL and UL HARQ processes increases significantly (e.g., 'disc_errors' reaching 1396, 'miss_errors' reaching 100).
25    * **Interpretation**: This confirms the BLER trend. The system is struggling to reliably acknowledge and retransmit data, suggesting interference or fading is impacting the control channels (PDCCH/PUSCH).
26 3. **PUSCH DTX**: This metric (missed PUSCH detections) also rises, indicating difficulty in reliably conveying ACK/NACK feedback, which is critical for HARQ retransmissions.
27
28 #### ⚠️ Timing and Synchronization Warnings
29 1. **Invalid timing advance offset for RNTI ffd4**: This warning appears repeatedly and is a critical indicator.
30    * **Interpretation**: Timing Advance (TA) is used to correct the timing offset between the UE and the gNB. Repeated 'Invalid timing advance offset' warnings mean the UE is either unable to accurately measure its timing offset or the gNB is receiving
31    timing reports that are inconsistent or outside acceptable bounds. This can severely limit the achievable UL data rate and stability.
32
33 #### 🚨 MAC Layer Anomalies
34 1. **Received unknown MAC header (LCID = 0x3f) / 'Received unknown MAC header (LCID = 0x3f)': These warnings indicate that the UE is receiving MAC PDUs with logical channel IDs (LCIDs) that are not expected or configured.
35    * **Interpretation**: While the MAC layer is designed to discard unknown RLC/LCIDs per the context documentation, frequent unknown headers can point to interference, neighboring cell leakage, or misconfigurations in the network stack.
36 2. **Residual UL MAC PDU is 117.7 with length = 0; pdu_len -15730**: This is a severe internal processing error, suggesting a buffer overflow or data corruption issue within the MAC layer processing of the PDU.
37
38 ---
39 ## 🛠️ Suggested Mitigation Strategies
```

No causal chain has been build. Different parts being explained separately. No link between them.

Mode 7: Causal chain built with targeted prompt but no RAG:

```
## 3. Causal Chain Analysis

The root cause (timing misalignment) initiates a cascade of failures that degrade the connection quality, leading to the observed instability and eventual shutdown.

| Stage | Event/Error | Description | Impact on Connection Quality |
| :--- | :--- | :--- | :--- |
| **Initial State** | Successful RA/Connection | The UE successfully completes the Random Access procedure ('CBRA procedure succeeded:') and establishes initial synchronization. | High (Initial connection established). |
| **Root Cause Trigger** | **Invalid timing advance offset for RNTI ffd4** | The UE's timing mechanism begins to drift or miscalculate the necessary timing advance offset. | **Critical**: Data transmitted by the UE is physically misaligned in time, causing the gNB to receive corrupted or unusable signals. |
| **Level 1 Failure (MAC/PHY)** | **residual UL MAC PDU... | Because the timing is wrong, the received bits are corrupted. The MAC layer attempts to parse the corrupted data, resulting in a protocol parsing failure. | **Severe**: Data integrity is lost. The connection cannot reliably transfer large amounts of data. |
| **Level 2 Failure (MAC/Protocol)** | **Received unknown MAC header | The corruption is so severe that the MAC layer cannot even identify the data format, treating the payload as unknown or malformed. | **Severe**: The protocol stack cannot process the incoming data stream, leading to discarded packets. |
| **Level 3 Failure (Application/Scheduler)** | **UE ffd4: Detected UL Failure on PUSCH... | The cumulative effect of timing errors, corrupted PDUs, and unknown headers means the UE's scheduler repeatedly fails to confirm successful data transmission (PUSCH). | **Critical**: The UE assumes the link is unusable and stops attempting to transmit data, effectively dropping the connection. |
| **Final State** | **Caught SIGTERM, shutting down | The system detects the persistent failure (UL Failure) and terminates the connection/thread. | **Failure**: Connection loss.
```

Explanation: RAG does not provide LLM with the information to do deep causal analysis. Only gives information on definitions of MAC entities.

5. Classical NLP (scoring using hardcoded vocabulary, scoring using vocabulary learned from Chi2) based methods and sentence embedding based methods are better at extracting key events from the logs.

Evidence to support: Evidence for this has already been provided in observation 3 above. On the file, harq_exhaustion_logs_new_trimmed.txt, almost all modes where classical NLP based event retrieval is used, the events retrieved are the relevant ones. On the other hand, LLM based event retrieval has multiple HARQ RETXs at the top (see point 3).

Explanation: Same as in point 3.

6. RAG provides almost no improvement to key event extraction based methods.

Regardless of whether RAG is implemented over the key event extraction methods, the results are the same. This tells us that RAG based on specs + docs offers no information to perform deep root cause analysis of logs.

Below is a comparison of causal chains obtained from Mode 4 (Static+no_rag) and Mode 5 (Static+rag_concat)

```
## Causal Chain

The root cause ('Invalid timing advance offset') initiates a cascade of physical and logical errors, severely degrading the connection quality over time.

### Phase 1: Physical Layer Degradation (The Root Cause)
* **Root Cause:** 'Invalid timing advance offset for RNTI ffd4'
* **Immediate Effect:** The gNB cannot accurately time-align the incoming uplink signals from the UE.
* **Symptom 1:** The UE's uplink transmissions (PUSCH/UL-SCH) are received with timing misalignment.

### Phase 2: Synchronization and Measurement Errors
* **Impact:** Because the timing is off, the physical layer processing struggles to correctly decode the signal parameters.
* **Symptom 2:** 'UE ffd4: reported RSRP index -> Invalid' (The UE is reporting invalid measurements, likely due to the timing issue corrupting the measurement data).
* **Symptom 3:** 'Received unknown MAC header (LCID = <*>)' (The timing misalignment and corrupted physical layer data can cause the MAC layer to misinterpret the start or structure of the incoming packet, leading to unknown headers).

### Phase 3: Connection Quality Degradation (The Observable Impact)
* **Impact:** The cumulative effect of timing errors and corrupted packets leads to increased error rates and reduced throughput.
* **Metric Degradation:**
  * **BLER (Block Error Rate):** The BLER values show a clear trend of increasing instability, especially in the later slots (e.g., moving from 0.00000$ to 0.12974$ and 0.14367$). This indicates that a significant portion of the transmitted data is corrupted or lost.
    * **UL-RI (Uplink Received Interference):** While the value remains 1, the associated error rates confirm the instability.
    * **MAC/LCID Data:** The overall throughput (TX/RX bytes) remains high, suggesting the connection is *trying* to maintain service, but the increasing BLER shows that the quality of the data transfer is rapidly deteriorating.

### Phase 4: System Shutdown (The Final Outcome)
* **Impact:** The persistent, unresolvable physical layer errors eventually trigger higher-level failure mechanisms.
* **Symptom:** The logs show the UE detecting UL failures and eventually the system receiving a 'SIGTERM' signal, leading to the graceful shutdown of the radio processing threads ('[PHY] Stopping RU 0 processing threads', 'Bye.').
```

Mode 4

```
### Causal Chain

**1. Root Cause (PHY Layer Failure):**
* **Event:** 'Invalid timing advance offset for RNTI ffd4'
* **Mechanism:** The UE (ffd4) is unable to maintain or report a valid timing advance offset to the gNB.
* **Immediate Effect:** The gNB's ability to correctly decode the timing of the UE's uplink transmissions (PUSCH) is compromised.

**2. Secondary Impact (MAC/RLC Layer Degradation):**
* **Observation:** The logs show repeated instances of this error occurring across multiple slots (e.g., Slot 128.0, 256.0, 384.0, etc.).
* **Effect:** The MAC layer, while attempting to process the PUSCH data, is constantly receiving timing warnings. Although the log shows the MAC layer *receiving* the data (e.g., 'UE ffd4: MAC: TX 76521083 RX 1918699 bytes'), the underlying timing instability means that the data integrity is questionable, and the system is operating under stress.

**3. Tertiary Impact (Performance Metrics Deterioration):**
* **Observation:** The logs show the BLER (Block Error Rate) fluctuating and increasing significantly in the later slots:
  * *Early Slots:* BLER $approx 0.00000$ (Excellent)
  * *Mid Slots:* BLER $approx 0.12$ to 0.20$ (Poor)
  * *Late Slots:* BLER remains high (e.g., 0.14367, 0.16916, 0.11688).
* **Mechanism:** The timing instability (Root Cause) leads to increased packet loss and decoding errors, which directly manifests as a high and unstable **BLER**.
* **Observation:** The 'dl_sch_errors' (DL SCH errors) and 'ul_sch_errors' (UL SCH errors) also increase dramatically, indicating that the MAC layer is struggling to maintain reliable data transfer.

**4. System Response (Mitigation/Failure):**
* **Observation:** The logs show the system attempting to recover or re-initialize the connection (e.g., the RA procedure for 'ad5d' and the subsequent 'Remove NR rnti 0xffd4').
* **Effect:** The repeated failures force the system to perform complex recovery procedures (like re-running RA or removing the UE context), which consume resources and indicate a persistent, unresolved connection quality issue.
```

Mode 5

Limitations and Failure Analysis

Discuss system limitations and bottlenecks.

Evaluation Criteria: Honesty, depth of reflection, and understanding of system constraints.

Limitations and Bottlenecks:

1. RAG done using specs + OAI documentation. This is good for the purpose for question-answering. For deep root cause analysis, it would be more appropriate to collect documents dedicated to 5G fault analysis. The document must contain effects of each log anomaly.
2. The sentence embedding and clustering assumes that anomalies are rare, extracting key events by picking the logs from the smallest clusters. However, if a specific MAC failure triggers a massive, cascading log storm (e.g., a broadcast storm or thousands of rapid HARQ retransmission failures), the resulting cluster will be large. The current logic would bypass this massive cluster, completely missing the actual catastrophic failure.
3. Significant words in Static Miner are hardcoded. New anomalies would require this dictionary to be expanded. Dynamic Miner was developed in this project to address this exact limitation.
4. Does not do cross-layer diagnosis (across PHY, MAC, RRC, etc.)

Evaluation Against Objectives

Table mapping objectives to outcomes and evidence.

Evaluation Criteria: Alignment between goals and outcomes with clear evidence.

Objective	Outcome	Evidence
1. Mitigate prompt dependency from 5G MAC log analysis.	Achieved	Replaced manual prompt engineering with automated anomaly extraction and prompt enrichment. Evidence: <code>scripts/full_pipeline.py,</code> <code>src/mine_logs.py,src/mine_logs_dynamic.py,</code> <code>src/sentence_clustering.py</code> All outputs containing causal chains are present in <code>results/<log_filename>/mode<num>/</code> where <code>num=4,5,6,7,8,9,10,11,12,13</code>
2. Explore the use of RAG in 5G MAC log analysis and assess its efficacy.	Achieved	Used Hybrid BM25+Vector Search RAG ground LLM responses in spec context. RAG was found to be ineffective. Evidence:

Objective	Outcome	Evidence
		<i>scripts/ingestion_pipeline.py, src/retrieval_pipeline.py</i> Outputs in <i>results/<log_filename>/mode<num></i> where <i>num=1,2,3</i> show that RAG has no impact analysis quality. Outputs in <i>results/<log_filename>/mode<num></i> where <i>num=5,6,8,9,11,12</i> show that RAG has no effect on the causal chain produced with key event extraction.
3. Explore alternative data-driven approaches to close the operator skill gap.	Achieved	Developed three distinct, autonomous extraction methods: a Static Miner (Drain3 heuristic scoring), a Dynamic Miner (Chi-Square statistical vocabulary learning), and Sentence Clustering (Agglomerative isolation of semantic anomalies), and an LLM-based event extraction. Evidence: <i>scripts/full_pipeline.py, scripts/mode13_pipeline.py, src/mine_logs.py, src/mine_logs_dynamic.py, src/sentence_clustering.py</i> Artifacts corresponding to
4. Compare and contrast the different approaches and assess their efficacy.	Achieved	Executed a 13-mode comparative pipeline showing that RAG is not useful for RCA and that event extraction and targeted prompting is far more effective. Evidence: All the code files in the <i>scripts/ and src/</i> directories. The outputs in the <i>results/</i> directory.

Demo Description

Describe demo scenario and observations.

Evaluation Criteria: Clarity and relevance of demonstration.

Demo Scenario: Will run the 13-mode pipeline and show all produced artifacts.

Work Distribution

Roles, contributions, % effort verified through GitHub repo commits.

Evaluation Criteria: Fairness, transparency, and accountability.

Roles: All work individually done by me.

Conclusions

Summary and impact.

Evaluation Criteria: Clarity and ability to synthesize contributions.

Summary:

The project compares the following systems:

- Baseline LLM (gemma4)
- Baseline LLM + Concat RAG
- Baseline LLM + LLM Translated RAG
- Static Mining + no RAG
- Static Mining + Concat RAG
- Static Mining + LLM Translated RAG
- Dynamic Mining + no RAG
- Dynamic Mining + Concat RAG
- Dynamic Mining + LLM Translated RAG
- Sentence Clustering + no RAG
- Sentence Clustering + Concat RAG
- Sentence Clustering + LLM Translated RAG
- LLM Key Event Extraction + no RAG

All systems above except for the first three use targeted prompting + key event extraction

The first three use a generic prompt with RAG (no RAG in the first one).

The first three systems are unable to build a meaningful causal chain.

All other systems can build a causal chain because of targeted prompting with appropriate events given as seeds.

The use of RAG does not impact any system in particular. Therefore, RAG on 3GPP specs (i.e., TelecomRAG) is not appropriate for this problem.

What is effective is building prompts. Better strategies to extract anomalous events from MAC logs would lead to better prompts and therefore, better root cause analyses.

Future Work

Extensions and research/deployment opportunities.

Evaluation Criteria: Creativity and feasibility of future directions.

Future Work:

1. Instead of using specs for RAG, use older logs and their corresponding root cause analyses. That is, use RAG to add historical context.
2. LLM based key event extraction was weak. Efforts can be dedicated to developing a specialized event extraction framework. Instruction fine tuning can be done here. Will require at least 1000 log files over varied scenarios with corresponding anomalies identified (ground truth). This is a data collection exercise. Feasible on a testbed.
3. Cross-layer analysis. The current system only looks at MAC layer logs in isolation. Future work should focus on developing agents to do cross-layer analysis. Perhaps using individual agents for each layer of the protocol stack and using a central agent to obtain aggregate analysis.

References

Citations and GitHub repo link.

Evaluation Criteria: Proper citation and completeness.

Citations:

1. *LLM-5GMAC: Performance Optimization in O-RAN Split 7.2 Using LLM-Based MAC-Layer Log Analysis* (<https://dl.acm.org/doi/10.1145/3737900.3770164>)
2. *TelecomRAG: Taming Telecom Standards with Retrieval Augmented Generation and LLMs* (<https://dl.acm.org/doi/10.1145/3711992.3711996>)

Github Repo Link: <https://github.com/dhanraj-s/TWiN-IITH-5GMACLogAnalysis>

Declaration on LLM Usage

LLMs used, prompts, validation, documentation.

Evaluation Criteria: Transparency and responsible AI usage.

Used LLMs to integrate individual scripts into a clean pipeline utilizing config files for setting parameters, implementing caching to save API credits, and building new scripts by extending the code in prior scripts.

After writing the rough scripts that would save directly in the working directory, would use hardcoded paths, etc. used Gemini to couple all those scripts in a single workflow utilizing config.json for setting paths, and model names and saving the outputs cleanly into the results/ directory.

Appendix: GitHub Repository Structure & Submission Guidelines

1. Repository Naming Convention

Use format: TWiN-IITH-<ProjectShortName>

2. Recommended Repository Structure

TWiN-Project/
docs/ (report, slides, diagrams)
src/ (core code)
configs/ (config files)
experiments/ (scripts)
results/ (outputs)
scripts/ (utilities)
demo/ (demo/video)
README.md
requirements.txt/environment.yml
LICENSE (optional)

3. README.md (Compulsory)

Include:

- Project overview
- Setup instructions
- How to run
- Experiment reproduction steps
- Results summary
- LLM usage disclosure

4. Code Quality Expectations

Modular code, meaningful naming, comments, avoid hardcoding (use configs)

5. Reproducibility Requirements

TA should be able to clone, run, and reproduce at least one key result

6. Version Control Best Practices

Use meaningful commits, avoid unnecessary files, use .gitignore

7. Submission on Moodle

Submit GitHub link in report PDF and slides. Ensure repo access is enabled for TAs and instructors. My GitHub ID: btamma

8. Evaluation Criteria

Structure, reproducibility, code quality, documentation, contribution evidence